

[Bài đọc] Giới thiệu @ApiControllerAdvice và @ExceptionHandler

1. Mở đầu

- Trong quá trình phát triển ứng dụng **RESTful API** với Spring Boot, việc xử lý ngoại lệ (exception) là một phần quan trọng nhằm đảm bảo ứng dụng hoạt động ổn định, dễ bảo trì và mang lại trải nghiệm tốt cho người dùng. Nếu không có cơ chế tập trung xử lý lỗi, các đoạn code xử lý ngoại lệ sẽ bị rải rác trong từng controller, gây khó khăn trong quản lý và dễ dẫn đến lỗi trùng lặp
- Để giải quyết vấn đề này, Spring Boot cung cấp hai công cụ mạnh mẽ:
 - **@RestControllerAdvice**: Cho phép tập trung xử lý ngoại lệ cho toàn bộ ứng dụng ở một lớp duy nhất
 - **@ExceptionHandler**: Cho phép định nghĩa phương thức xử lý riêng cho từng loại ngoại lệ cụ thể

2. @RestControllerAdvice là gì?

- Là một annotation trong Spring Boot, đóng vai trò như một **“trung tâm điều phối lỗi”**
- Thay vì xử lý lỗi tại từng controller, ta gom toàn bộ về một lớp duy nhất để quản lý
- Giúp code trở nên **gọn gàng, dễ bảo trì, tránh trùng lặp**

3. @ExceptionHandler là gì?

- Được sử dụng bên trong lớp có gắn @RestControllerAdvice
- Mỗi phương thức đánh dấu @ExceptionHandler sẽ **xử lý một loại ngoại lệ cụ thể**
- Ví dụ:
 - **MethodArgumentNotValidException** → dữ liệu request không hợp lệ (400 Bad Request)
 - **NoSuchElementException** → dữ liệu không tồn tại (404 Not Found)
 - **Exception** chung → lỗi server nội bộ (500 Internal Server Error)

4. Lợi ích khi kết hợp hai annotation này

- **Chuẩn hóa:** Tất cả phản hồi lỗi được định nghĩa theo một cấu trúc đồng nhất (`ApiResponse`, `ApiError...`)
- **Dễ bảo trì:** Thay đổi logic xử lý lỗi chỉ cần chỉnh tại một nơi
- **Chính xác:** Trả về mã HTTP phù hợp với từng tình huống (400, 404, 500...)
- **Trải nghiệm tốt hơn cho client:** Client dễ dàng xử lý và hiển thị lỗi

5. Bảng biểu so sánh

Tiêu chí	@RestControllerAdvice	@ExceptionHandler
Mục đích	Tập trung xử lý ngoại lệ toàn ứng dụng	Xử lý một loại ngoại lệ cụ thể
Phạm vi áp dụng	Toàn bộ các REST controller	Trong một phương thức xử lý riêng
Cách sử dụng	Gắn annotation vào lớp xử lý lỗi chung	Gắn annotation vào phương thức xử lý lỗi
Ví dụ thực tế	Lớp <code>GlobalExceptionHandler</code> xử lý mọi lỗi API	Phương thức <code>handleNotFound()</code> xử lý 404
Lợi ích chính	Giúp mã nguồn gọn gàng, dễ bảo trì	Giúp xử lý chi tiết từng lỗi cụ thể

6. Kiến thức mở rộng

- Tạo cấu trúc phản hồi lỗi đồng nhất
 - Thay vì trả về dữ liệu lỗi ở nhiều định dạng khác nhau, ta nên dùng một **`ApiResponse`** hoặc **`ErrorResponse`** với các trường như:
 - **status** (thành công / thất bại)
 - **message** (mô tả lỗi)
 - **data** hoặc **errors** (chi tiết lỗi hoặc dữ liệu trả về)
 - Điều này giúp client dễ dàng xử lý và hiển thị thông tin một cách thống nhất
- Các mã lỗi HTTP thường dùng trong xử lý ngoại lệ
 - **400 Bad Request:** Dữ liệu request không hợp lệ
 - **404 Not Found:** Tài nguyên không tồn tại

- **401 Unauthorized / 403 Forbidden:** Lỗi xác thực hoặc phân quyền
- **500 Internal Server Error:** Lỗi phía server
- **429 Too Many Requests:** Quá nhiều request trong thời gian ngắn (rate limiting)
- Best Practices khi xử lý lỗi với `@RestControllerAdvice`
 - **Phân loại lỗi rõ ràng:** Không gom tất cả vào Exception chung, mà xử lý theo từng loại
 - **Trả về thông tin ngắn gọn, hữu ích:** Tránh lộ chi tiết hệ thống nội bộ
 - **Ghi log đầy đủ ở server:** Để tiện debug khi sự cố xảy ra
 - **Đồng bộ giữa backend và frontend:** Thống nhất cấu trúc phản hồi lỗi để frontend dễ parse dữ liệu